

Atividade 01 — PoC de Retrieval Vetorial

Promessa ou Contradição? — GenAI & RAG

Objetivo

Nesta primeira atividade da frente de **GenAI & RAG**, o objetivo será construir uma prova de conceito simples para validar a etapa de **Vector Search** do projeto.

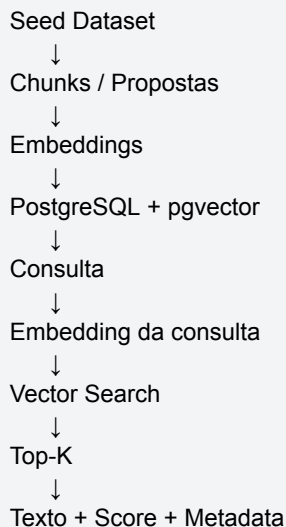
A ideia é partir de um pequeno conjunto de trechos/propostas, gerar embeddings, armazená-los em um banco com PostgreSQL + pgvector e, a partir de uma consulta, recuperar os Top-K trechos semanticamente mais relacionados.

Nesta etapa, o foco é apenas em retrieval.

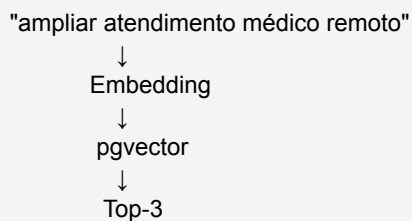
Não é necessário implementar ainda:

- RAG completo;
- classificação com LLM;
- Keyword Search;
- Hybrid Search;
- métricas de avaliação.

Arquitetura esperada



Exemplo:



Resultado:

```
1. chunk_03
  score: 0.91
  página: 14
  seção: Saúde
  texto: "Expandir serviços de telemedicina"

2. chunk_08
  score: 0.84
  ...

3. chunk_12
  score: 0.72
  ...
```

O que fazer

1. Criar um Seed Dataset

Criar um dataset pequeno com aproximadamente 15 a 20 trechos/propostas.

Cada item deve possuir pelo menos:

```
{
  "chunk_id": "chunk_01",
  "document_id": "document_01",
  "page": 10,
  "section": "Saúde",
  "text": "Expandir o atendimento médico remoto para municípios do interior."
}
```

Também podem ser adicionados como metadata:

```
candidate
party
office
state
```

Para esta PoC, os exemplos podem ser fictícios.

2. Criar exemplos interessantes

O dataset deve possuir casos pensados para testar a busca.

Caso 1 — Textos semelhantes

Trecho A:
"Expandir serviços de telemedicina."

Trecho B:
"Ampliar o atendimento de saúde por meio de consultas remotas."

Esperamos que os embeddings indiquem uma similaridade alta.

Caso 2 — Textos não relacionados

Trecho A:
"Expandir serviços de telemedicina."

Trecho B:
"Construir novas linhas ferroviárias."

Esperamos uma similaridade menor.

Caso 3 — Palavras semelhantes, mas ideias diferentes

Trecho A:
"Reduzir impostos para pequenas empresas."

Trecho B:
"Manter os atuais impostos cobrados das pequenas empresas."

Esse caso é importante para observar que **similaridade semântica não significa necessariamente que duas propostas sejam equivalentes ou concordem entre si.**

A interpretação dessa relação será feita posteriormente pelo LLM.

3. Gerar os embeddings

Cada texto deve ser transformado em um embedding.

Fluxo:

```
Texto
↓
Embedding Model
↓
Vetor
```

Exemplo:

```
"Expandir serviços de telemedicina"
↓
Embedding Model
↓
[0.12, -0.34, 0.76, ...]
```

O modelo utilizado deve ser identificado no projeto.

4. Armazenar no PostgreSQL + pgvector

Persistir no banco:

```
chunk_id
document_id
page
section
text
embedding
```

Fluxo:

```
Seed Dataset
↓
Python
↓
Embeddings
↓
PostgreSQL + pgvector
```

A metadata deve continuar associada ao texto após a vetorização.

5. Implementar a busca Top-K

Criar uma busca que receba uma consulta, por exemplo:

```
"melhorar o atendimento médico remoto"
```

Fluxo:

```
Consulta
↓
Embedding
↓
Comparação com os embeddings armazenados
↓
Vector Search
↓
Top-K
```

Inicialmente utilizar:

```
Top-K = 3
```

Mas o valor deve ser configurável.

Exemplo de resultado esperado

```
QUERY: melhorar o atendimento médico remoto
```

```
TOP-3 RESULTADOS
```

```
1. chunk_04 | score: 0.91 | página: 12 | seção: Saúde
```

"Expandir serviços de telemedicina para municípios do interior."

2. chunk_09 | score: 0.84 | página: 20 | seção: Saúde

"Criar atendimento médico remoto para regiões de difícil acesso."

3. chunk_15 | score: 0.63 | página: 7 | seção: Saúde

"Ampliar a rede de unidades básicas de saúde."

Testes mínimos

Realizar pelo menos 3 consultas.

Consulta 1

Uma consulta muito parecida com algum trecho existente.

Consulta 2

Uma consulta semanticamente semelhante, mas utilizando palavras diferentes.

Consulta 3

Uma consulta mais difícil ou ambígua.

Em cada caso, observar:

O resultado esperado apareceu?

Em qual posição?

O score de similaridade fez sentido?

A metadata foi preservada?

Entrega

A entrega deve conter:

- Seed Dataset;
- geração de embeddings;
- PostgreSQL + pgvector;
- ingestão dos dados;
- Vector Search;
- Top-K configurável;
- score de similaridade;
- metadata dos resultados;
- demonstração de pelo menos 3 consultas.

Resultado esperado

Ao final da atividade, devemos conseguir demonstrar:

Consulta
↓
Embedding
↓
pgvector
↓
Top-K
↓
Trechos relacionados
+
Score
+
Origem

A atividade estará concluída quando conseguirmos afirmar:

“Dada uma consulta ou proposta, conseguimos recuperar os Top-K trechos semanticamente mais relacionados e identificar de qual documento, página e seção cada resultado veio.”

Essa PoC será a base para as próximas etapas:

Vector Search
↓
Keyword Search
↓
Hybrid Search
↓
Evaluation
↓
RAG
↓
Análise Semântica